

SE Master Project Proposal Template

Nick Glas

23rd April 2026

Project Details

Project Title:	Secure and Efficient Neural Network Inference Across Microservices
Student:	Nick Glas
Academic Supervisors:	dr. Ana Oprescu, a.m.oprescu@uva.nl, CCI
Host	University of Amsterdam – Complex CyberInfrastructure
Start Date:	1 April 2026
Planned End Date:	31 July 2026

1 Introduction

Neural networks are increasingly used in a wide range of applications, including computer vision, natural language processing, and decision support systems. As these models grow in size and complexity, the computational cost of running inference has increased significantly. At the same time, many applications operate on sensitive data or rely on proprietary models, which introduces strong privacy and security requirements during inference.

In this context, software systems are increasingly adopting microservices architectures to improve scalability, maintainability, and deployment flexibility. This architectural style decomposes applications into smaller independently deployable services that communicate over a network. Such architectures have become especially relevant in cloud and edge computing environments.

Neural network inference using microservice architectures is an attractive direction because it allows inference pipelines to be deployed, scaled, and managed across distributed infrastructure. However, decomposing inference across service boundaries introduces new challenges. In particular, service boundaries may increase end-to-end latency through serialization, remote procedure call overhead, and inter-service communication, while also increasing the number of locations at which sensitive data or model information may be exposed.

In existing studies, inference efficiency and inference security are largely treated as separate concerns. Research on inference efficiency primarily focuses on reducing latency and communication overhead through techniques such as neural network partitioning, compression of intermediate activations, and communication-aware execution strategies [6, 8, 9, 11]. In contrast, work on inference security mainly concentrates on protecting the confidentiality of input data and model parameters by securing inference frameworks, for example through cryptographic protocols or trusted execution environments, which typically introduces additional computational and communication overhead [2, 7, 10, 16, 17]. Importantly, much of this prior work assumes execution models that differ from cloud-native microservice-based inference, such as tightly coupled edge-cloud deployments, shared execution environments, or hardware-level partitioning. As a result, these approaches do not explicitly account for the performance implications of independently deployed inference components communicating through standard RPC mechanisms and later being extended into containerized multi-microservice deployments.

As a consequence, the question remains as to how neural network inference can be decomposed across service boundaries in a way that balances efficiency and security. This gives rise to the central research question of this thesis:

How can neural network inference be decomposed across multiple microservices in a way that maintains strong security guarantees while minimizing performance overhead, as measured by inference latency and communication-related cost?

This question is investigated through a staged experimental design. First, literature-informed and architecturally valid decomposition choices are evaluated in a controlled

local setting using two-part inference splits in order to measure end-to-end latency and communication-related overhead. Starting with a single split provides the smallest distributed inference configuration that still exposes the core trade-off between computation placement and boundary-crossing cost, while avoiding the additional confounding effects introduced by multiple service boundaries. Within this controlled local stage, the thesis also includes a supporting internal timing study of the monolithic ResNet-18 forward pass in order to characterize compute distribution across stages, residual blocks, and selected internal operations. This auxiliary study is not treated as a separate split-selection experiment, but as explanatory evidence that helps interpret the partitioning behavior observed in the two-part experiments. Next, the study is extended to chained multi-microservice deployment in Kubernetes to examine the effect of increasing the number of service boundaries under a fixed execution model. Finally, selected configurations are validated on Azure, where their performance behavior is reassessed and where the security direction of the thesis is evaluated.

To further structure this investigation, the following active sub-research questions are considered:

- **RQ1** How do literature-informed sequential decomposition choices for ResNet-18 affect latency and communication-related overhead when inference is executed across microservices?
- **RQ2** Which security mechanisms are most suitable for distributed neural network inference across microservices, and what overhead and system complexity do they introduce when evaluated in an Azure deployment, including a trusted execution environment direction on Azure?

RQ3 from earlier drafts is deferred and is not part of the active scope of this thesis.

1.1 RQ1 Structure and Sub-Research Questions

RQ1 is decomposed into a set of literature-grounded sub-research questions to support a controlled and interpretable evaluation. Prior work indicates that partitioning outcomes depend on where boundaries are placed, the granularity of the split, the balance of computation across execution units, and the cost of transferring intermediate activations across service boundaries. Building on this direction, this thesis studies sequential cut-based decomposition choices for ResNet-18 in a microservice-style inference setting through the following sub-questions.

1. **RQ1.1** Which coarse architectural stage boundaries provide the most suitable two-part microservice decompositions of ResNet-18 under controlled local execution? (Partition ResNet-18 using a single split after each major architectural stage and compare each coarse decomposition against monolithic execution using latency and communication-related measurements.)
2. **RQ1.2** How does finer-grained refinement within the late-stage coarse region carried forward from RQ1.1 affect latency and communication-related overhead for two-part microservice inference? (Select the most suitable non-degenerate late-stage coarse region from RQ1.1 and evaluate finer-grained split points within

that carried-forward region to determine how refined boundary placement changes the latency–communication trade-off.)

3. **RQ1.3** How do activation-transfer size and compute distribution explain the behavior observed for the decomposition choices evaluated in RQ1.1 and RQ1.2? (Measure intermediate activation sizes and the compute time associated with each side of the partition in order to explain the latency and communication behavior observed in the coarse and refined two-part decomposition stages. To strengthen the compute-distribution explanation, a supporting internal timing study is also performed on the monolithic ResNet-18 forward pass to characterize how compute is distributed across stages, residual blocks, and selected internal operations.)
4. **RQ1.4** How does increasing the number of microservices affect end-to-end inference cost when ResNet-18 is decomposed into synchronously chained coarse architectural stages under in-cluster Kubernetes execution? (Extend beyond two-part partitioning by deploying inference as an in-cluster chained microservice pipeline in Kubernetes, where one ingress request enters the first service and intermediate activations are forwarded synchronously from one coarse-stage segment to the next until inference is completed, and measure how additional service boundaries affect end-to-end latency and communication-related overhead.)
5. **RQ1.5** To what extent do the service-count trends observed for the same synchronously chained coarse-stage decompositions under local in-cluster Kubernetes execution remain consistent when those configurations are deployed on Azure AKS? (Select representative in-cluster chained configurations from the local Kubernetes experiments and deploy them on Azure to measure whether the latency and communication-related trends observed locally remain directionally consistent in a cloud environment with different network and platform characteristics.)

Taken together, RQ1.1 through RQ1.3 establish the controlled local partition-screening, refinement, and explanation stage. Within this local stage, the internal timing study is used only as supporting explanatory evidence for compute distribution and not as a replacement for split benchmarking. RQ1.4 extends this analysis to true multi-microservice Kubernetes deployment under a chained execution model, and RQ1.5 provides the Azure validation step for the selected chained configurations.

1.2 RQ2 Structure and Sub-Research Questions

RQ2 is structured as a progressive security-hardening stage that builds directly on the Azure deployment path established in RQ1.5. Whereas RQ1 focuses on identifying, refining, explaining, and validating suitable microservice decomposition choices, RQ2 examines how selected Azure-based distributed inference configurations behave when security protections are incrementally introduced. The purpose of this stage is not to repeat the partition-selection process, but to evaluate how stronger security guarantees affect latency, communication-related cost, and deployment complexity within the already selected deployment context.

This design follows a progressive hardening logic. First, the selected Azure deployment

from RQ1.5 is retained as the non-secure baseline. Next, inter-service communication is secured using mutual TLS and service identity mechanisms in order to protect intermediate activations in transit and strengthen authentication between inference services. Finally, a trusted execution environment direction is introduced on Azure in order to protect sensitive remote execution and reduce exposure of intermediate data and model computation within untrusted cloud infrastructure. By evaluating these hardening steps incrementally, the thesis can isolate the additional cost introduced by communication protection separately from the cost introduced by runtime protection.

1. **RQ2.1** What overhead and deployment complexity are introduced when inter-service communication is secured using mutual TLS and service identity mechanisms on the selected Azure-based distributed inference configuration? (Starting from the non-secure Azure baseline established in RQ1.5, apply communication-level protection to the selected distributed inference pipeline and measure the resulting effect on end-to-end latency, communication-related overhead, tail latency, and deployment complexity.)
2. **RQ2.2** What additional overhead and deployment complexity are introduced when a trusted execution environment direction is added to the communication-secured Azure-based distributed inference configuration? (Extend the communication-protected Azure deployment with a trusted execution environment direction for selected remote inference components, and measure the incremental impact on latency, communication-related cost, and system complexity relative to the already secured Azure baseline.)
3. **RQ2.3** Which security-hardening level provides the most suitable trade-off between security guarantees, performance overhead, and deployment complexity for distributed neural network inference on Azure? (Compare the non-secure Azure baseline, the communication-secured deployment, and the communication-secured plus trusted-execution deployment in order to determine which security level offers the most credible balance between stronger protection and acceptable system cost.)

Taken together, RQ2.1 through RQ2.3 form a staged security evaluation pipeline built on the Azure deployment context established in RQ1.5. RQ2.1 isolates the cost of protecting service-to-service communication, RQ2.2 evaluates the additional cost of protecting remote execution through a trusted execution environment direction, and RQ2.3 synthesizes the resulting trade-offs in order to identify the most suitable security-hardening level for the evaluated distributed inference setup.

2 Related Work

Following the guidelines of Webster and Watson [19], this section reviews the literature in a concept centric manner. Rather than discussing individual papers in isolation, prior work is synthesized around the key ideas relevant to this proposal: (i) partitioned and distributed neural network inference and its latency communication trade-offs, (ii) security threats and protection mechanisms for decomposed inference, and (iii) microservice deployments and performance abstractions. The goal is to identify which desired features for efficient and secure inference are addressed by existing research and which gaps remain.

2.1 Partitioned and Distributed Neural Network Inference

A substantial body of work explores how neural network inference can be partitioned across multiple nodes or tiers (e.g., devices, edge or cloud) to reduce end to end latency under resource constraints. Common approaches perform layer or block wise partitioning and search for split points that balance local computation time against the cost of transmitting intermediate activations [6, 8, 9, 11]. Across these studies, a consistent finding is that communication overhead is highly sensitive to the size of intermediate feature maps and available bandwidth, often dominating inference latency when early layers are offloaded.

To mitigate this effect, several systems combine partitioning with compression or adaptive strategies that respond to changing network conditions [4, 8, 13]. Other studies explore structural transformations or model parallel execution to reduce synchronization and communication overhead [12]. While these studies provide valuable insights into latency communication trade offs, they largely assume tightly coupled execution environments (e.g., edge cloud links) and do not explicitly consider microservice style execution with RPC stacks, containerization, and service isolation.

2.2 Security Threats and Protection Mechanisms for Inference

When neural network inference is decomposed into multiple components, the threat surface increases. Intermediate activations may leak information about inputs or model parameters, and each execution boundary introduces risks such as interception, tampering, or unauthorized access. Privacy preserving inference research demonstrates that strong cryptographic guarantees can be achieved using techniques such as secure multi party computation or homomorphic encryption, but typically at significant cost in latency and communication overhead [2, 7, 10]. These approaches establish important baselines, showing that full cryptographic protection is often impractical for latency sensitive inference.

Trusted Execution Environments (TEEs) provide an alternative by protecting code and data at runtime. Previous studies show that TEEs can reduce the attack surface but introduce overhead due to memory constraints, paging, and enclave transitions [3, 16, 17]. In parallel, distributed and privacy aware inference systems begin to address security concerns in multi node settings, yet often rely on assumptions and

communication models different from cloud-native microservices [5, 15, 18]. Collectively, this literature highlights that security mechanisms introduce measurable performance overhead and system complexity, motivating careful evaluation when inference is decomposed.

2.3 Microservices and AI Inference Deployment

Microservice-based AI and edge computing literature typically treats neural networks as black-box services and focuses on orchestration, placement, and quality-of-service optimization rather than on the internal structure of inference execution [14, 15, 18]. These systems commonly model latency at the level of service invocations and optimize deployment or scheduling decisions accordingly, but they abstract away the internal contributors to inference latency, such as layer-wise computation, serialization, and intermediate activation transfer [14, 15]. As a result, the interaction between neural network internals and microservice execution boundaries remains underexplored. In contrast, benchmarking and layer-level performance analysis provide detailed insights into inference behavior within neural networks [1, 6, 8, 9]. However, these studies are generally conducted in monolithic, tightly coupled, or edge-cloud execution settings and do not account for the additional overhead introduced by microservice communication stacks and service-level security mechanisms.

Benchmarking and layer level performance analysis provide methodological foundations for understanding inference behavior [1]. However, these studies are generally conducted in monolithic or tightly coupled settings and do not account for the additional overhead introduced by microservice communication stacks and security policies.

2.4 Research Gap

Across these themes, existing work delivers important individual features: communication aware partitioning of inference [6, 8], strong privacy guarantees through cryptographic or trusted execution mechanisms [10, 16], and scalable deployment through microservice orchestration [14]. However, there is limited systematic analysis at their intersection. In particular, prior work does not empirically decompose inference latency across microservice boundaries into computation, serialization, and network transfer, nor does it quantify how existing security mechanisms affect these costs and overall system complexity in a staged local-to-cloud evaluation workflow. This gap motivates the active research questions on partitioning strategy evaluation and security overhead in multi-microservice neural network inference.

2.5 Concept Matrix

Table 1 summarizes representative related work and highlights the research gap.

Work	Partitioning	Comm. analysis	Security	Microservices
DNNSplit [6]	✓	✓	×	×
DeeperThings [9]	✓	✓	×	×
CNNPC [8]	✓	✓	×	×
JointCompressPartition [11]	✓	✓	×	×
DeepSlicing [4]	✓	✓	×	×
OfpCNN [13]	✓	✓	×	×
DecoupledCNN [12]	✓	✓	×	×
GAZELLE [10]	×	✓	✓	×
CHEETAH [2]	×	✓	✓	×
EncryptedLargeDNN [7]	×	✓	✓	×
SLALOM [16]	◦	◦	✓	×
DarkneTZ [3]	◦	◦	✓	×
TEESlice [17]	◦	◦	✓	×
Delphi [5]	◦	✓	✓	×
RLDistPrivacy [15]	✓	✓	✓	×
PPStream [18]	✓	✓	✓	×
HephaestusForge [14]	×	◦	◦	✓
BackboneAnalysis [1]	×	◦	×	×
This thesis	✓	✓	✓	✓

Table 1: Matrix of representative related work (✓ explicit, ◦ partial, × not addressed).

3 Methodology

This section describes the methodology used to answer the research questions of this thesis. The thesis adopts an experimental systems research paradigm, which involves a targeted literature review as well as the design and evaluation of a proof-of-concept neural network inference system based on a microservice architecture. This system enables the evaluation of inference behavior under distributed execution through a staged experimental design. The methodology begins with controlled local evaluation of two-part partitioning strategies, then extends to multi-microservice deployment in Kubernetes, and finally proceeds to Azure-based validation and security assessment.

3.1 Research Challenges

Investigating neural network inference decomposed across service boundaries involves several methodological and technical challenges. A key challenge lies in disentangling the individual contributors to end-to-end inference latency. When neural network inference is partitioned across services, execution time depends not only on computation within each partition, but also on the serialization and deserialization of intermediate activations, RPC overhead, network transmission effects, and later microservice orchestration overhead. These factors interact in complex ways. As a result, attributing observed performance effects to a single cause becomes difficult without carefully designed baselines and controlled experiments.

A second challenge concerns the intrinsic variability of distributed execution environments. Even in local settings, process scheduling, background system activity, and runtime effects can inject noise into latency measurements, particularly when trying to discern fine-grained performance differences between partitioning strategies. This challenge becomes more pronounced when the study later moves to Kubernetes, where container scheduling and network contention add further sources of variability. Statistically meaningful and reproducible results consequently require repeated measurements using consistent deployment configurations.

Analysis is further complicated by the need to evaluate security mechanisms alongside performance. Techniques such as encrypted inter-service communication and trusted execution environments introduce overheads that depend on message size, model architecture, and execution context. In this project, the trusted execution environment direction is evaluated on Azure, which adds platform-specific constraints and deployment decisions to the measurement process. Security mechanisms can also change system behavior in subtle ways, for example by affecting batching efficiency or memory usage. It is non-trivial to design experiments that fairly compare secure and non-secure configurations while maintaining a coherent threat model.

Another challenge involves choosing representative models of neural networks and strategies for partitioning. Candidate partitions must be selected from architecturally valid boundaries and informed by prior literature, rather than generated arbitrarily. While complex models may be more representative of real-world workloads, their internal optimizations and hardware-specific behavior make effects involving distribution and security mechanisms harder to isolate. On the other hand, overly simple models risk

limiting the external validity of findings. Balancing realism with experimental control is therefore crucial.

Finally, the existing literature on distributed inference and secure neural network execution has often assumed tightly coupled edge-cloud or hardware-level execution environments rather than microservice-based architectures. As a result, performance insights from prior work do not directly apply to the staged workflow adopted here, in which controlled local two-part partition screening and refinement are followed by multi-microservice Kubernetes deployment and later Azure validation.

3.2 Experiments and Proof of Concept

The experimental component of this project is centered around the development of a proof-of-concept system for microservice-based neural network inference. The proof of concept uses ResNet-18 as the main experimental model and supports three closely related evaluation modes: monolithic execution, controlled two-part partitioning across a single service boundary, and later synchronously chained multi-microservice execution across selected coarse architectural segments. In addition, the local stage includes a separate internal timing mode that profiles the monolithic ResNet-18 forward pass in order to characterize compute distribution across stages, residual blocks, and selected internal operations. This internal timing mode is exploratory and explanatory in purpose: it is used to strengthen the interpretation of partitioning results rather than to replace the split-benchmarking experiments.

Proof of Concept System The proof of concept begins with a baseline configuration that executes the complete model within a single process. It is then extended to support controlled two-part partitioning, in which inference is split across a single service boundary with one local segment and one remote segment communicating through a standard RPC interface. Later, the system is extended to support deployment across multiple containerized microservices under a synchronous chained execution model. Finally, the same Azure-based deployment path is progressively hardened with communication security and trusted execution mechanisms in order to evaluate the performance and complexity cost of security protections within a consistent distributed inference setting. The platform is designed so that candidate decomposition choices, execution environments, and later security mechanisms can be varied while keeping the rest of the system as constant as possible.

Partition-Selection Methodology for RQ1 Partition candidates are not generated through exhaustive blind search. Instead, candidates are constructed from architecturally valid model boundaries, beginning with coarse stage-level boundaries and then refining only the carried-forward late-stage region with finer-grained block-level boundaries. Candidate construction is guided by literature on distributed inference, where latency depends on the balance between local computation and intermediate activation transfer. This approach yields a representative set of plausible sequential decomposition choices that can be compared in a controlled way. Additional internal

timing analysis may be used to characterize compute concentration within the network, but such profiling is treated as explanatory support rather than as the primary basis for selecting split points.

Experiments for RQ1.1–RQ1.3: Controlled Local Partition Screening, Refinement, and Explanation

The first experimental stage is conducted in a controlled local environment. This stage focuses on monolithic execution and two-part partitioning across a single service boundary. RQ1.1 evaluates coarse architectural split points, RQ1.2 refines the most suitable non-degenerate coarse region using finer-grained split points, and RQ1.3 explains the observed performance differences using activation-transfer size and compute distribution. The primary outcomes in this stage are end-to-end inference latency and communication-related overhead, supported by intermediate measurements such as activation sizes and per-part compute times. In addition, a supporting internal timing experiment is conducted on the monolithic ResNet-18 forward pass to profile compute distribution across stages, blocks, and selected internal operations. This auxiliary experiment is used to strengthen the compute-distribution explanation in RQ1.3 by showing where local computation is actually concentrated, but it is not treated as a substitute for the split-benchmark results.

Experiments for RQ1.4: Multi-Microservice Kubernetes Stage

After the controlled local screening and refinement stage, the study is extended to true multi-microservice deployment in Kubernetes. This stage addresses RQ1.4 by evaluating how increasing the number of service boundaries affects end-to-end inference cost, latency, and communication overhead under a containerized deployment setting. The execution model in this stage is explicitly synchronous and chained: one ingress request enters the first service, and intermediate activations are forwarded from one coarse-stage segment to the next until inference is completed.

Experiments for RQ1.5: Azure Validation Stage

Based on the local results, a smaller set of representative partitioning strategies is selected and deployed on Azure. This stage addresses RQ1.5 by testing whether key partitioning trends remain consistent outside the controlled local environment. The same core outcomes, latency and communication overhead, are measured again to support a direct local-to-cloud comparison. In this stage, Azure is treated as the validated deployment context on which the security-focused evaluation of RQ2 is later built.

Experiments for RQ2: Progressive Security Hardening on Azure

To investigate the second research question, the study builds directly on the Azure-based distributed inference configurations established in RQ1.5 rather than reopening the partition-selection process. The purpose of this stage is to evaluate how stronger security guarantees affect latency, communication-related overhead, and deployment complexity within the already selected Azure deployment path.

The RQ2 evaluation follows a progressive hardening workflow. First, the non-secure Azure deployment serves as the baseline configuration. Second, inter-service

communication is secured using mutual TLS and service identity mechanisms in order to protect intermediate activations in transit and strengthen authentication between inference services. Third, the communication-secured deployment is extended with a trusted execution environment direction on Azure in order to protect sensitive remote execution and reduce exposure of model computation and intermediate representations in untrusted cloud infrastructure.

This staged design allows the thesis to separate the cost of communication protection from the additional cost of runtime protection. The resulting impact of each hardening step is measured relative to the corresponding non-secure or less-secure Azure baseline. The primary outcomes are end-to-end inference latency, communication-related overhead, and system complexity, where system complexity includes the additional deployment, configuration, and operational burden introduced by each security mechanism.

Experiments for RQ2.1: Communication Security on Azure The first security stage evaluates the overhead and deployment complexity introduced when the selected Azure-based distributed inference configuration is secured using mutual TLS and service identity mechanisms. This stage isolates the effect of protecting service-to-service communication and authenticated service interaction while keeping the deployment topology otherwise fixed. Measurements focus on the additional latency and communication-related overhead introduced by communication protection, together with the practical complexity of configuring certificates, identities, and related deployment artifacts.

Experiments for RQ2.2: Trusted Execution Direction on Azure The second security stage evaluates the additional overhead and deployment complexity introduced when a trusted execution environment direction is added to the already communication-secured Azure deployment. This stage focuses on protecting selected remote inference components during execution and on quantifying the incremental cost of stronger runtime protection relative to the communication-secured baseline. Measurements again focus on latency, communication-related cost, and the additional engineering and operational complexity introduced by the trusted execution setup.

Experiments for RQ2.3: Security Trade-off Synthesis The final RQ2 stage synthesizes the results of the non-secure Azure baseline, the communication-secured deployment, and the trusted-execution-hardened deployment. The goal is to determine which hardening level provides the most suitable balance between stronger security guarantees, acceptable performance overhead, and manageable deployment complexity for the evaluated distributed inference pipeline. This final step does not introduce a new deployment mode, but instead compares the earlier RQ2 stages as alternative security levels within the same Azure-based inference context.

The previously considered third experiment stream is deferred and is not part of the active experimental scope of this plan.

3.3 Research Method

This research uses an experimental systems research methodology. It involves the design, implementation, and experimental evaluation of a proof-of-concept system for microservice-based neural network inference. Controlled experiments and comparative analysis are used throughout the system development process. A monolithic inference deployment serves as a baseline for comparison with partitioned inference configurations. Experimental evaluation is performed by varying selected system parameters, such as literature-informed candidate partitioning strategies and the inclusion of security mechanisms, while keeping other system variables fixed. This approach enables the analysis of the impact of these variables on system latency, communication cost, and overall system efficiency.

For RQ1, the analysis is structured around architectural boundaries, partition granularity, activation-transfer and compute-distribution behavior, and microservice count. The evaluation proceeds in three stages. First, a controlled local stage is used to evaluate coarse two-part partitioning boundaries and finer-grained refinements within the most suitable coarse region. Second, the study is extended to Kubernetes to examine the impact of increasing the number of microservices under a true multi-microservice deployment. Third, representative configurations are evaluated on Azure for validation under a cloud environment. To account for variability inherent in distributed systems, experiments are conducted over multiple trials and evaluated using aggregated system measurements.

The trusted execution environment direction is explicitly assessed in the Azure stage as part of the security-focused research question. The chosen methodology directly supports the investigation of the system-level research questions posed in this thesis.

3.4 Hypotheses

Based on existing literature on distributed neural network inference, secure computation, and microservice architectures, this project formulates the following hypotheses.

Partitioning Overhead Decomposing neural network inference across microservices increases end-to-end inference latency compared to monolithic execution, primarily due to inter-service communication and serialization overhead.

Partitioning Strategy Sensitivity The impact of microservice-based inference on latency and communication overhead depends strongly on the chosen candidate partitioning strategy, with strategies aligned to meaningful architectural boundaries outperforming arbitrary or overly fine-grained splits.

Activation-Transfer and Compute Balance Differences between candidate partitioning strategies are expected to be explained by the balance between local computation and intermediate activation-transfer size, where larger transferred feature maps are associated with higher communication overhead and increased latency.

Microservice Count Effect Increasing the number of microservices beyond a moderate level is expected to increase end-to-end inference cost, because additional service boundaries add serialization, RPC, and coordination overhead.

Cross-Environment Consistency Representative partitioning strategies that perform well in the local evaluation stages are expected to remain relatively favorable during Azure validation, although absolute latency and communication overhead values may shift due to cloud networking and platform effects.

Security-Induced Performance Cost Applying security mechanisms to inter-service communication in microservice-based inference introduces additional latency and resource overhead, with the magnitude of this overhead depending on the mechanism type and on whether an Azure-based trusted execution environment is used.

3.5 Timeline and Research Flow

The thesis is planned over a period of approximately four months and is structured into sequential phases, where the outcome of each phase informs the next.

Phase 1: Literature Consolidation The project begins with a focused literature study on neural network partitioning, secure inference, and microservice-based execution. The outcome of this phase is a refined understanding of existing approaches, relevant performance metrics, and open gaps, which directly inform the design of the proof-of-concept system and experimental methodology.

Phase 2: System Design and Controlled Local Platform Setup Based on insights from the literature, a proof-of-concept inference system is designed and implemented. This phase results in a configurable platform that supports both monolithic execution and controlled two-part partitioning. Completion of this phase enables systematic experimentation for RQ1.1–RQ1.3.

Phase 3: Local Partition Screening and Refinement (RQ1.1–RQ1.3) Using the implemented platform, controlled local experiments are conducted to evaluate coarse architectural boundaries, finer-grained refinements within the selected coarse region, and the explanatory role of activation-transfer size and compute distribution. Measurements of inference latency and communication overhead establish the main local partitioning results and identify representative strategies.

Phase 4: Multi-Microservice Kubernetes Experiments (RQ1.4) Building on the earlier local results, the study is extended to Kubernetes to evaluate inference across more than two microservices. This phase measures how additional service boundaries affect latency and communication overhead in a containerized deployment.

Phase 5: Azure Validation of Partitioning (RQ1.5) Representative partitioning strategies from the earlier stages are deployed on Azure. This phase validates whether findings from the local stages remain consistent in cloud deployment and quantifies Azure-specific latency and communication behavior.

Phase 6: Security Experiments on Azure (RQ2) Security mechanisms are introduced into the Azure deployment, including a trusted execution environment direction on Azure. Experiments in this phase quantify the additional overhead introduced by security measures and assess their interaction with selected partitioning strategies.

Phase 7: Analysis and Thesis Writing The final phase focuses on consolidating experimental results, validating findings, and writing the thesis. Results from all active phases are synthesized to answer RQ1 and RQ2, and to discuss implications, limitations, and deferred future work.

3.6 Validation

The validity of the research is ensured through a combination of controlled experimentation, comparative evaluation, and transparent reporting of assumptions and limitations.

First, all experimental results are validated against a monolithic inference baseline, which serves as a reference point for assessing the impact of partitioned execution. This comparison ensures that observed performance effects can be attributed to distribution and security mechanisms rather than implementation artifacts.

Second, validation follows a staged process. Candidate partitioning strategies are first evaluated in a controlled local setting through coarse two-part screening and finer-grained refinement, then extended to a multi-microservice Kubernetes deployment, and finally re-evaluated on Azure. This design checks whether relative performance trends observed under controlled local conditions remain meaningful as the deployment context becomes more realistic.

Third, experiments are conducted using repeated measurements under consistent system configurations to reduce the influence of transient effects such as background load, scheduling variability, and deployment-specific noise. Where appropriate, summary statistics such as median and tail latency percentiles are reported to capture both typical and worst-case behavior.

Fourth, validation focuses on internal consistency across experiments. Observed trends in latency and communication overhead are cross-checked against intermediate measurements, such as activation sizes and per-partition execution times, to ensure that conclusions align with underlying system behavior.

Fifth, security validation is performed on Azure and explicitly includes the trusted execution environment direction on Azure. Overheads are interpreted relative to non-secure Azure deployments to keep comparisons fair within the same platform context.

Finally, the scope and limitations of the study are explicitly acknowledged. The evaluation is conducted using a limited set of candidate partitioning strategies and

security mechanisms applied to a representative convolutional neural network model, which constrains generalizability. The deferred third research direction is treated as future work rather than part of the current validation scope.

4 Risk Assessment

This section identifies potential risks associated with the project and outlines mitigation strategies to reduce their likelihood or impact. Table 2 summarizes the main risks, their estimated likelihood and impact, and corresponding contingency measures.

Description	Risk			Action
	Likelihood	Impact	Score	
Scope creep due to excessive model or configuration choices	4	5	20	Restrict evaluation to a single CNN model and a limited set of partitioning strategies and security mechanisms
Complexity of decomposing CNN models into independently deployable microservices	3	4	12	Start with a simple, well-documented CNN architecture; validate layer-level execution incrementally before introducing distributed communication
Experimental setup complexity (microservices, networking, and security integration)	3	4	12	Incremental system development beginning with a monolithic baseline and gradually introducing partitioning and security features
Performance variability in microservice-based execution environments	3	3	9	Use repeated measurements, fixed deployment configurations, and controlled experimental conditions
Misinterpretation of performance measurements due to system variability	2	3	6	Apply established benchmarking practices and cross-check results using intermediate metrics such as activation sizes and per-service execution times
Limited availability or suitability of hardware resources	2	3	6	Design experiments to run on commonly available hardware and avoid reliance on specialized infrastructure
Project planning or time estimation errors	3	3	9	Maintain weekly progress tracking and preserve buffer time for analysis and writing
Supervisor availability constraints	1	2	2	Schedule regular check-ins early and communicate progress and questions proactively

Table 2: Risk Register

5 Ethics

An ethics self-assessment was conducted via the Research Management Services (RMS) portal of the Faculty of Science. This assessment evaluated potential ethical considerations related to data usage, involvement of human participants, privacy, and the use of artificial intelligence technologies.

The outcome of the self-assessment indicates that the proposed study does not involve the active participation of human subjects. The research does not involve the collection, use, or analysis of sensitive human data or personal data. No existing datasets containing information about identifiable individuals are used. All data generated and analyzed in this study consists of synthetic performance measurements obtained from controlled experiments conducted within the proof-of-concept system.

Although the research concerns artificial intelligence technologies, its focus lies on system-level performance analysis and experimental evaluation rather than on the development or deployment of AI systems that directly interact with users or real-world decision-making processes. Based on the ethics self-assessment, no significant ethical risks were identified within the scope of this project. No further action from the Ethical Review Committee is required at this stage.

6 Project Plan

This section presents the project plan and milestones for the thesis. The project is divided into a sequence of phases that correspond to the research questions and experimental steps described in the methodology. Progress is defined in terms of achieved results rather than activities. Table 3 summarizes the planned timeline.

Period	Milestone (Achieved Result)
Weeks 1–3	Literature study completed and experiment plan defined, including partition candidate selection criteria, selected metrics, and security mechanisms.
Weeks 4–6	Monolithic CNN inference baseline and controlled local proof-of-concept platform implemented, supporting both monolithic execution and two-part partitioning for RQ1.1–RQ1.3.
Weeks 7–9	Controlled local partition screening and refinement completed for RQ1.1–RQ1.3; coarse architectural boundaries, finer-grained refinements, and explanatory activation-transfer and compute-distribution measurements analyzed.
Weeks 10–11	Multi-microservice Kubernetes experiments completed for RQ1.4, measuring the effect of increasing the number of service boundaries on latency and communication overhead.
Weeks 11–12	Representative partitioning strategies deployed to Azure and validated with the same core performance metrics to address RQ1.5.
Weeks 12–13	Security mechanisms evaluated on Azure, including a trusted execution environment direction, with measured performance and complexity overhead (RQ2).
Weeks 8–16	Thesis written, revised based on experimental results and supervisor feedback, and finalized for submission.

Table 3: Project timeline with milestones expressed as achieved results.

References

- [1] “BackboneAnalysis: Structured Insights into Compute Platforms from CNN Inference Latency”. In: *ACM SIGMETRICS Performance Evaluation Review* (2021).
- [2] “CHEETAH: Lean and Fast Secure Two-Party Deep Neural Network Inference”. In: *USENIX Security Symposium*. 2020.
- [3] “DarkneTZ: Towards Model Privacy at the Edge using Trusted Execution Environments”. In: *ACM MobiSys*. 2020.
- [4] “DeepSlicing: Collaborative and Adaptive CNN Inference with Low Latency”. In: *IEEE Internet of Things Journal* (2020).
- [5] “Delphi: A Cryptographic Inference Service for Neural Networks”. In: *USENIX Security Symposium*. 2019.
- [6] “DNNsSplit: Latency and Cost-Efficient Split Point Identification for Multi-Tier DNN Partitioning”. In: *IEEE Transactions on Mobile Computing* (2021).
- [7] “Enabling Homomorphically Encrypted Inference for Large DNN Models”. In: *Proceedings of Machine Learning and Systems* (2022).
- [8] “End-Edge-Cloud Collaborative CNN Inference with Joint Model Partition and Compression”. In: *Future Generation Computer Systems* (2022).
- [9] “Fully Distributed CNN Inference on Resource-Constrained Edge Devices”. In: *Cluster Computing* (2021).
- [10] “GAZELLE: A Low Latency Framework for Secure Neural Network Inference”. In: *USENIX Security Symposium*. 2018.
- [11] “Joint Compressing and Partitioning of CNNs for Fast Edge-Cloud Collaborative Intelligence”. In: *Journal of Systems Architecture* (2022).
- [12] “Model Parallelism Optimization for Distributed Inference via Decoupled CNN Structure”. In: *IEEE INFOCOM*. 2020.
- [13] “On-Demand Fine-Grained Partitioning for CNN Inference Acceleration in Heterogeneous Devices”. In: *IEEE Transactions on Mobile Computing* (2020).
- [14] “Optimal Microservice Deployment Across the Compute Continuum via Reinforcement Learning”. In: *Future Generation Computer Systems* (2024).
- [15] “Privacy-Aware Distributed Deep Inference for Low-Latency IoT Systems”. In: *IEEE Internet of Things Journal* (2022).
- [16] “SLALOM: Fast, Verifiable and Private Execution of Neural Networks in Trusted Hardware”. In: *International Conference on Learning Representations*. 2019.
- [17] “TEESlice: Protecting Sensitive Neural Network Models in Trusted Execution Environments”. In: *arXiv preprint* (2023).
- [18] “Toward High-Performance Privacy-Preserving Neural Network Inference via Distributed Stream Processing”. In: *IEEE Transactions on Dependable and Secure Computing* (2023).

- [19] J. Webster and R. T. Watson. “Analyzing the Past to Prepare for the Future: Writing a Literature Review”. In: *MIS Quarterly* 26.2 (2002), pp. xiii–xxiii.